Лекция 8: Поведенческие паттерны GoF. Strategy, Observer, Command, State, Chain of Responsibility

Курс: Объектно-ориентированное программирование 1 (CS2203)
Университет: КазНУ им. аль-Фараби
Преподаватель: Санжар Аубакиров, PhD
Аудитория: Бакалавриат, 2-й курс, специальность «6B06110 Программная инженерия»
Формат: Лекция, 2 академических часа (90 минут)
Методологический нарратив: Order → Emergence (От структуры объектов к динамическому взаимодействию систем)

Введение и инженерный контекст

Мы изучили, как создавать объекты (порождающие паттерны) и как компоновать их в устойчивые структуры (структурные паттерны).
Сегодня мы переходим к третьему семейству паттернов GoF — **поведенческим паттернам (Behavioral Patterns)**. Поведенческие шаблоны отвечают за **алгоритмы, распределение ответственности и передачу управления между объектами во времени**.
В реальной распределенной системе (такой как терминалы и серверы Avtobys) объекты постоянно обмениваются сообщениями в динамике: - Терминал переходит между состояниями (ожидание карты → чтение NFC → обработка → сигнал успеха); - Каждая транзакция проходит строгую цепочку валидаций (проверка стоп-листа → антифрод → проверка баланса); - Диспетчерская служба и аналитика должны мгновенно реагировать на события без жесткой привязки к валидаторам; - Операции должны уметь становиться в очередь для автономного режима (Offline mode), когда в степях под Алматы пропадает 4G-связь.
Все эти задачи элегантно решаются поведенческими паттернами: **Strategy**, **Observer**, **Command**, **State** и **Chain of Responsibility**.

План лекции (90 минут)

№	Время	Тема	Формат
1	00:00–00:20	Паттерн Стратегия (Strategy): семейство взаимозаменяемых алгоритмов	Сравнение решений
2	00:20–00:40	Паттерн Наблюдатель (Observer / Pub-Sub): слабая связанность и реакция на события	Теория + код

№	Время	Тема	Формат
3	00:40–01:05	Паттерн Команда (Command): инкапсуляция запроса как объекта (очереди и офлайн-режим)	Архитектурный разбор
4	01:05–01:25	Паттерны Состояние (State) и Цепочка обязанностей (Chain of Responsibility)	Практика
5	01:25–01:30	Резюме поведенческих паттернов и подготовка к лабораторной работе №8	Рефлексия

---

1. Паттерн Стратегия (Strategy)

Стратегия определяет семейство схожих алгоритмов, инкапсулирует каждый из них и делает их взаимозаменяемыми во время выполнения.
Вместо жесткого переключателя match / if-elif в классе контекста, алгоритм выносится в отдельный объект-стратегию. Мы уже сталкивались с этим при расчете скидок на Неделе 2.

```
# strategies/fraud.py
from typing import Protocol


class FraudCheckStrategy(Protocol):
    def is_suspicious(self, card_id: str, amount: float) -> bool: ...


class LenientFraudStrategy:
    """Мягкая стратегия (для утреннего часа пик — пропуск очереди)."""
    def is_suspicious(self, card_id: str, amount: float) -> bool:
        return False


class StrictFraudStrategy:
    """Строгая стратегия (ночные рейсы и крупные списания)."""
    def is_suspicious(self, card_id: str, amount: float) -> bool:
        return amount > 5000.0 or card_id.startswith("TEST")
```

---

2. Паттерн Наблюдатель (Observer / Publisher-Subscriber)

Наблюдатель создает механизм подписки, позволяющий одним объектам следить и реагировать на события, происходящие в других объектах, без жесткой зависимости.
В системе Avtobys валидатор не должен напрямую знать о телеграм-ботах, мониторинге Prometheus или базе данных аналитики. Валидатор просто публикует факт: «Транзакция завершена».

```
# patterns/observer.py
from typing import Protocol, List


class TransactionEvent:
    def __init__(self, tx_id: str, card_id: str, amount: float):
        self.tx_id = tx_id
        self.card_id = card_id
        self.amount = amount


class TransactionObserver(Protocol):
    def on_transaction_processed(self, event: TransactionEvent) -> None: ...


class ValidatorSubject:
    """Субъект (Издатель): уведомляет наблюдателей о транзакциях."""
    def __init__(self):
        self._observers: List[TransactionObserver] = []

    def attach(self, observer: TransactionObserver) -> None:
        self._observers.append(observer)

    def detach(self, observer: TransactionObserver) -> None:
        self._observers.remove(observer)

    def notify(self, event: TransactionEvent) -> None:
        for observer in self._observers:
            try:
                observer.on_transaction_processed(event)
            except Exception as e:
                print(f"[OBSERVER ERROR] {observer.__class__.__name__}: {e}")
```

---

3. Паттерн Команда (Command)

Команда превращает запросы или операции в самостоятельные объекты, позволяя передавать их как аргументы методов, ставить в очередь, логировать и поддерживать отмену (Undo).

Офлайн-режим валидатора в автобусе:

Когда автобус въезжает в тоннель или загородную зону, связь с процессинговым сервером пропадает. Валидатор не имеет права остановить посадку пассажиров! Он формирует объект команды ProcessPaymentCommand и сохраняет его в локальную

энергонезависимую очередь. При восстановлении сети очередь команд последовательно выполняется на сервере.

```
# patterns/command.py
from typing import Protocol


class Command(Protocol):
    def execute(self) -> bool: ...


class SyncTransactionCommand:
    def __init__(self, server_client, payload: dict):
        self._client = server_client
        self._payload = payload

    def execute(self) -> bool:
        return self._client.send_transaction(self._payload)
```

---

4. Паттерны Состояние (State) и Цепочка обязанностей (Chain of Responsibility)

4.1. Паттерн Состояние (State)

Позволяет объекту изменять свое поведение в зависимости от внутреннего состояния, как будто объект изменил свой класс.

Терминал валидатора работает как конечный автомат (FSM):

IdleState $\xrightarrow{\text{NFC detected}}$ ReadingState $\xrightarrow{\text{Success}}$ ApprovedState $\xrightarrow{\text{2 sec timeout}}$ IdleState

Вместо гигантского if self.state == "IDLE": ... elif self.state == "READING": ..., каждое состояние представляет собой отдельный класс с методами on_card_presented(), on_timeout(), on_error().

4.2. Паттерн Цепочка обязанностей (Chain of Responsibility)

Позволяет передавать запросы последовательно по цепочке обработчиков, где каждый обработчик решает: обработать запрос самостоятельно, прервать цепочку или передать запрос дальше.

```
[ Попытка оплаты ]
       |
       v
[ 1. CheckStopListHandler ] ----(В стоп-листе)----> [ Отклонить: Карта заблокирована ]
       | (OK)
       v
[ 2. AntiFraudIntervalHandler ] -(< 60 сек)------> [ Отклонить: Повторное прикладывание ]
       | (OK)
       v
[ 3. SufficientFundsHandler ] --(Баланс < Тарифа)-> [ Отклонить: Недостаточно средств ]
       | (OK)
       v
[ 4. ExecutionHandler ] ------------------------> [ УСПЕХ: Средства списаны ]
```

Каждый узел цепочки абсолютно изолирован и отвечает ровно за одно бизнес-правило (строгое соблюдение SRP и OCP).

Контрольные вопросы и выводы

1. В чем ключевое различие между паттернами Strategy и State, если их UML-диаграммы классов почти идентичны?
2. Почему в паттерне Observer важно изолировать сбои в вызовах конкретных наблюдателей от субъекта?
3. Каким образом паттерн Command позволяет реализовать отказоустойчивый офлайн-режим валидации?
4. Какие преимущества дает разбиение сложной финансовой валидации на Цепочку обязанностей (Chain of Responsibility)?

Литература к лекции

1. **Гамма Э. и др.** Паттерны проектирования. – СПб.: Питер, 2020. (Глава 5: Поведенческие паттерны).
2. **Фаулер М.** Шаблоны корпоративных приложений. – М.: Вильямс, 2016.
3. **Refactoring.Guru**: Поведенческие паттерны (https://refactoring.guru/ru/design-patterns/behavioral-patterns).